第二章

线性表

主要内容：

2.1 线性表的类型定义

2.2 线性表的顺序表示和实现

2.3 线性表的链式表示和实现

2.4 一元多项式的表示及相加

线性表是一种最简单的线性结构

线性结构的基本特征为:线性结构是一个数据元素的有序（次序）集

线性结构的基本特征为:

1. 集合中必存在唯一的一个“第一元素”；
2. 集合中必存在唯一的一个“最后元素”；
3. 除最后元素在外，均有唯一的后继；
4. 除第一元素之外，均有唯一的前驱。

§ 2.1
线性表的类型定义

抽象数据类型线性表的定义如下：

ADT List {

数据对象：

$D = \{ a_i \mid a_i \in ElemSet, i=1,2,\ldots,n, \ n \geq 0 \}$

{称 **n** 为线性表的表长;

称 **n=0** 时的线性表为空表。}

数据关系：

$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, \ i=2,\ldots,n \}$

{设线性表为 $(a_1, a_2, \ldots, a_i, \ldots, a_n)$,

称 i 为 a_i 在线性表中的位序。}

基本操作：

结构初始化操作

结构销毁操作

引用型操作

加工型操作

} **ADT** List

初始化操作

InitList(&L)

操作结果：

构造一个空的线性表L。

结构销毁操作

DestroyList(&L)

初始条件：线性表 L 已存在。

操作结果：销毁线性表 L。

引用型操作：

ListEmpty(L)

ListLength(L)

PriorElem(L, cur_e, &pre_e)

NextElem(L, cur_e, &next_e)

GetElem(L, i, &e)

LocateElem(L, e, compare( ))

ListTraverse(L, visit( ))

ListEmpty(L)
（线性表判空）

初始条件：线性表L已存在。

操作结果：若L为空表，则返回TRUE，否则FALSE。

ListLength(L)

（求线性表的长度）

初始条件：线性表**L**已存在。

操作结果：返回**L**中元素个数。

PriorElem(L, cur_e, &pre_e)

（求数据元素的前驱）

初始条件：线性表L已存在。

操作结果：若cur_e是L的元素，但不是第一个，则用pre_e 返回它的前驱，否则操作失败，pre_e 无定义。

NextElem(L, cur_e, &next_e)

（求数据元素的后继）

初始条件：线性表L已存在。

操作结果：若cur_e是L的元素，但不是最后一个，则用next_e返回它的后继，否则操作失败，next_e无定义。

GetElem(L, i, &e)

（求线性表中某个数据元素）

初始条件：线性表**L**已存在，

且 **1≤i≤LengthList(L)**。

操作结果：用 e 返回**L**中第 i 个元素的值。

LocateElem(L, e, compare( ))
（定位函数）

初始条件：线性表**L**已存在，e为给定值，compare( )是元素判定函数。

操作结果：返回**L中第1个**与**e满足**关系compare( )的元素的位序。若这样的元素不存在，则返回值为0。

ListTraverse(L, visit( ))

（遍历线性表）

初始条件：线性表**L**已存在，
Visit() 为某个访问函数。

操作结果：**依次**对**L**的每个元素调用
函数visit( )。一旦visit( )失败，
则操作失败。

加工型操作

ClearList(&L)

PutElem(&L, i, &e)

ListInsert(&L, i, e)

ListDelete(&L, i, &e)

ClearList(&L)

（线性表置空）

初始条件：　线性表**L**已存在。

操作结果：　将**L**重置为空表。

PutElem(&L, i, &e)

（改变数据元素的值）

初始条件：线性表L已存在，
且 $1 \le i \le LengthList(L)$ 。

操作结果：L中第i个元素赋值同e的值。

ListInsert(&L, i, e)

（插入数据元素）

初始条件：线性表**L**已存在，

且 **1≤i≤LengthList(L)+1** 。

操作结果：在**L**的第**i**个元素之前**插入**

新的元素e，L的长度增1。

ListDelete(&L, i, &e)

（删除数据元素）

初始条件：线性表L已存在且非空，

1≤i≤LengthList(L) 。

操作结果：删除L的第i个元素，并用e返回其值，L的长度减1。

利用上述定义的线性表，可以实现其它更复杂的操作

例 2-1

假设:有两个集合 A 和 B 分别用两个线性表 LA 和 LB 表示，即：线性表中的数据元素即为集合中的成员。

现要求一个新的集合A＝A∪B。

上述问题可演绎为：

要求对线性表作如下操作：

扩大线性表 LA，将存在于线性表 LB 中而不存在于线性表 LA 中的数据元素插入到线性表 LA 中去。

操作步骤：

1. 从线性表LB中依次察看每个数据元素；

   GetElem(LB, i)→e

2. 依值在线性表LA中进行查访；

   LocateElem(LA, e, equal( ))

3. 若不存在，则插入之。

   ListInsert(LA, n+1, e)

```
void union(List &La, List Lb) {
    La_len = ListLength(La);     // 求线性表的长度
    Lb_len = ListLength(Lb);
    for (i = 1;  i <= Lb_len;  i++) {
        GetElem(Lb, i, e); // 取Lb中第i个数据元素赋给e
        if (!LocateElem(La, e, equal( )) )
            ListInsert(La, ++La_len, e);
        // La中不存在和 e 相同的数据元素，则插入之
    }
} // union
```

- 时间复杂度为

$O(\text{ListLength(LA)} \times \text{ListLength(LB)})$

例 2-2

将非递减的有序表La 和Lb归并为新的线性表Lc, Lc的元素也按值非递减排列。

La=(3, 5, 8, 11)
Lb=(2, 6, 8, 9, 11, 15, 20)
Lc=(2, 3, 5, 6, 8, 8, 9, 11, 11, 15, 20)

```
void MergeList(List La, List Lb, List &Lc) {
  // 本算法将非递减的有序表La 和Lb 归并为Lc
  InitList(Lc); // 构造空的线性表Lc
  i = j = 1; k = 0;
  La_len = ListLength(La);
  Lb_len = ListLength(Lb);
  while ((i <= La_len) && (j <= Lb_len))
      { // La 和Lb 均不空}
  while (i<=La_len) // 若La 不空
  while (j<=Lb_len) // 若Lb 不空
} // MergeList
```

```
// La 和Lb 均非空，i = j = 1, k = 0
GetElem(La, i, ai);
GetElem(Lb, j, bj);
if (ai <= bj) {// 将ai 插入到Lc 中
    ListInsert(Lc, ++k, ai); ++i; }
else ListInsert(Lc, ++k, bj); ++j; }
```

```
while (i <= La_len) { // 当La不空时
    GetElem(La, i++, ai);
    ListInsert(Lc, ++k, ai);
} // 插入La 表中剩余元素
while (j <= Lb_len) { // 当Lb不空时
    GetElem(Lb, j++, bj);
    ListInsert(Lc, ++k, bj);
} // 插入Lb 表中剩余元素
```

算法的时间复杂度?

- 时间复杂度为

O(ListLength(LA)+ListLength(LB))

§ 2.2

线性表的顺序表示和实现

顺序映象

—— 以 x 的存储位置和 y 的存储位置之间某种关系表示逻辑关系<x, y>。

最简单的一种顺序映象方法是：

令 y 的存储位置和 x 的存储位置相邻。

用一组地址连续的存储单元依次存放线性表中的数据元素

	a_1	a_2	…	a_{i-1}	a_i	…	a_n	

线性表的起始地址

称作线性表的基地址

以“存储位置相邻”表示有序对$\langle a_{i-1}, a_i \rangle$

即：$LOC(a_i) = LOC(a_{i-1}) + C$

↑

每个数据元素所占存储量

所有数据元素的存储位置取决于

个数据元素的存储位置

$$LOC(a_i) = \underline{LOC(a_1)} + (i-1) \times C$$

↑基地址

线性表的顺序存储结构示意图

逻辑地址	数据元素
0	k_0
1	k_1
…	…
i	k_i
…	
n-1	k_{n-1}

存储地址	数据元素
$Loc(k_0)$	k_0
$Loc(k_0)+c$	k_1
…	…
$Loc(k_0)+i*c$	k_i
…	
$Loc(k_0)+(n-1)*c$	k_{n-1}

以元素在计算机内存中的“物理位置相邻”来表示线性表中数据元素之间的逻辑关系，只要确定了首地址，线性表中任意数据元素都可以随机存取。

顺序表示法的优缺点：

- **优点：**

1）存储密度大、空间利用率高

2）元素逻辑位置相邻，物理位置也相邻。可随机存取

- **缺点：**

1）插入和删除时要移动大量的元素

2）长度较大的线性表须按最大需要的空间分配存储空间

顺序映像的 C 语言描述

```
#define  LIST_INIT_SIZE     100
            // 线性表存储空间的初始分配量
#define  LISTINCREMENT    10
            // 线性表存储空间的分配增量
typedef  int  ElemType; //假定线性表元素的类型为整型
 typedef  struct {

    ElemType *elem;    // 存储空间基址

     int      length;   // 当前长度

     int      listsize;  // 当前分配的存储容量
                         // (以sizeof(ElemType)为单位)
 } SqList;  //  顺序表
```

- **申请内存函数**（malloc）

  malloc（ size ）

功能：申请长度为size字节的内存区。若申请成功，函数返回所分配的内存区首字节的地址，即指向该内存的指针；若申请失败，函数返回NULL。

说明：函数malloc的返回值为指向void类型的指针。这是通用指针类型，在实际申请内存空间时，要按照实际指针所指对象的类型进行指针类型强制转换。

- **申请内存函数**（realloc）

realloc（ p, size ）

功能：将p所指的已分配的内存区大小改为size。size可以比原来分配的空间大或小。若成功，函数返回所分配的内存区首字节的地址，即指向该内存的指针；若申请失败，函数返回NULL。

说明：函数的返回值为指向void类型的指针。这是通用指针类型，在实际申请内存空间时，要按照实际指针所指对象的类型进行指针类型强制转换。

- 释放内存（free）

free（ p ）

功能：释放p所指的内存空间。函数无返回值。这里p所指的内存区域必须是用函数malloc申请的内存空间，否则调用时使用其他指针，可能会破坏系统。

线性表的基本操作在顺序表中的实现

InitList(&L) // 结构初始化

LocateElem(L, e, compare()) // 查找

ListInsert(&L, i, e) // 插入元素

ListDelete(&L, i) // 删除元素

```
Status InitList_Sq( SqList& L ) {
  // 构造一个空的线性表
   L.elem = (ElemType*) malloc (LIST_
                    INIT_SIZE*sizeof (ElemType));
   if (!L.elem) exit(OVERFLOW);
   L.length = 0;
   L.listsize = LIST_INIT_SIZE
   return OK;
} // InitList_Sq
```

算法时间复杂度：*O*(1)

```
int LocateElem_Sq(SqList L, ElemType e,
  Status (*compare)(ElemType, ElemType)) {
  // 在顺序表中查询第一个满足判定条件的数据元素，
  // 若存在，则返回它的位序，否则返回 0
  i = 1;          // i 的初值为第 1 元素的位序
  p = L.elem;     // p 的初值为第 1 元素的存储位置
  while (i <= L.length &&
                  !(*compare)(*p++, e))  ++i;
  if (i <= L.length)  return i;
  else  return 0;
} // LocateElem_Sq
```

***O*(ListLength(L))**

线性表操作

ListInsert(&L, i, e) 的实现：

首先分析:

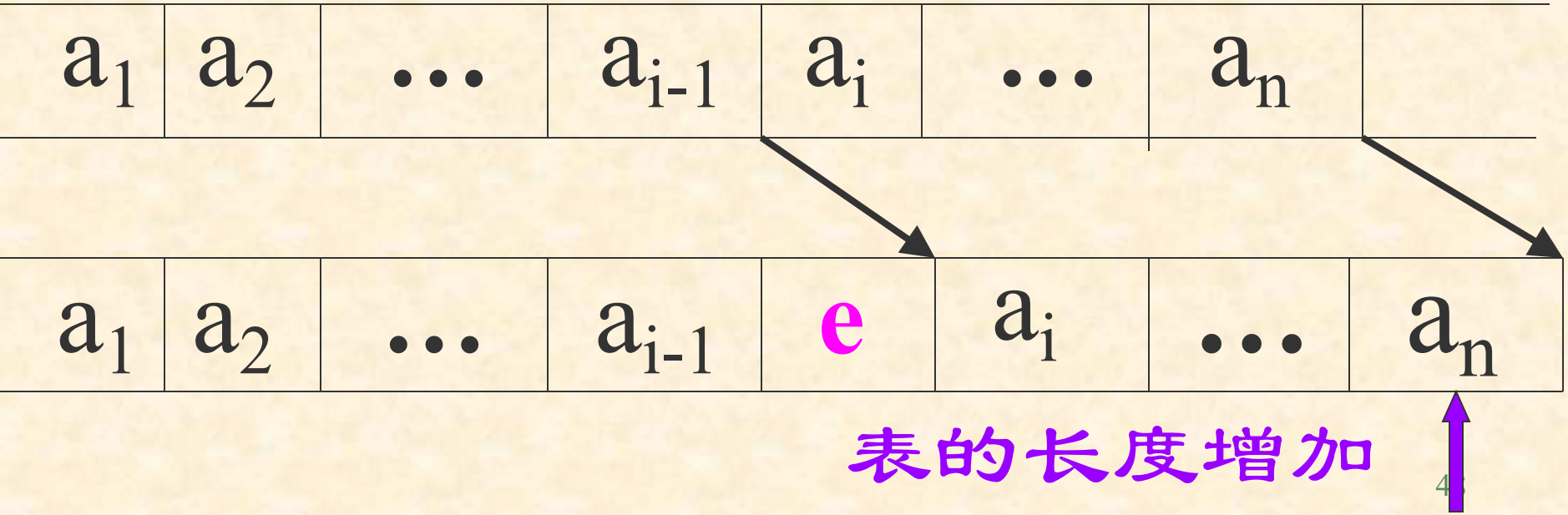
插入元素时，

线性表的逻辑结构发生什么变化？

$(a_1, \ldots, \mathbf{a_{i-1}}, \mathbf{a_i}, \ldots, a_n)$ 改变为

$(a_1, \ldots, \mathbf{a_{i-1}}, \mathbf{e}, \mathbf{a_i}, \ldots, a_n)$

$\langle \mathbf{a_{i-1}}, \mathbf{a_i} \rangle \longrightarrow \langle \mathbf{a_{i-1}}, \mathbf{e} \rangle, \langle \mathbf{e}, \mathbf{a_i} \rangle$



```
Status ListInsert_Sq(SqList &L, int i, ElemType e) {
 // 在顺序表L的第 i 个元素之前插入新的元素e,
 // i 的合法范围为 1≤i≤L.length+1
        ……
 q = &(L.elem[i-1]);                    // q 指示插入位置
 for (p = &(L.elem[L.length-1]); p >= q;  --p)
     *(p+1) = *p;        // 插入位置及之后的元素右移
 *q = e;        // 插入e
 ++L.length;   // 表长增1
 return OK;
} // ListInsert_Sq            O( ListLength(L) )
```

```
if (i < 1 || i > L.length+1) return ERROR;
                                    // 插入位置不合法
if (L.length >= L.listsize) {
                    // 当前存储空间已满，增加分配
  newbase = (ElemType *)realloc(L.elem,
    (L.listsize+LISTINCREMENT)*sizeof (ElemType));
  if (!newbase) exit(OVERFLOW);
                    // 存储分配失败
  L.elem = newbase;              // 新基址
  L.listsize += LISTINCREMENT; // 增加存储容量
}
```

考虑移动元素的平均情况：

假设在第i 个元素之前插入的概率为P_i，则在长度为n 的线性表中插入一个元素所需移动元素次数的期望值为：

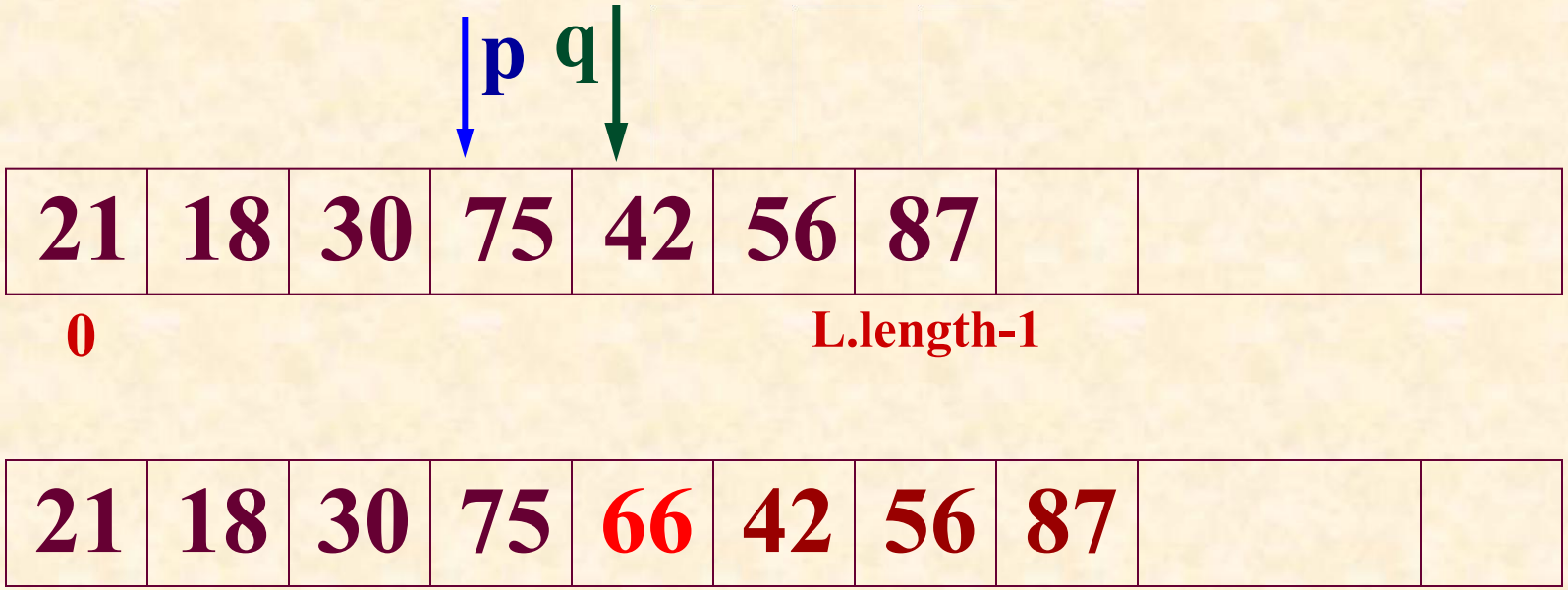
$$E_{is} = \sum_{i=1}^{n+1} p_i (n-i+1)$$

若假定在线性表中任何一个位置上进行插入的概率都是相等的，则移动元素的期望值为：

$$E_{is} = \frac{1}{n+1} \sum_{i=1}^{n+1} (n-i+1) = \frac{n}{2}$$

例如：ListInsert_Sq(L, 5, 66)

```
q = &(L.elem[i-1]);      // q 指示插入位置
for (p = &(L.elem[L.length-1]); p >= q;  --p)
    *(p+1) = *p;
```



线性表操作

ListDelete(&L, i, &e)的实现：

首先分析：

删除元素时，

线性表的逻辑结构发生什么变化？

$(a_1, \ldots, \mathbf{a_{i-1}}, \mathbf{a_i}, \mathbf{a_{i+1}}, \ldots, a_n)$ 改变为

$(a_1, \ldots, \mathbf{a_{i-1}}, \mathbf{a_{i+1}}, \ldots, a_n)$

$\langle \mathbf{a_{i-1}}, \mathbf{a_i} \rangle, \langle \mathbf{a_i}, \mathbf{a_{i+1}} \rangle \longrightarrow \langle \mathbf{a_{i-1}}, \mathbf{a_{i+1}} \rangle$

a_1	a_2	…	a_{i-1}	a_i	$\mathbf{a_{i+1}}$	…	a_n

a_1	a_2	…	a_{i-1}	$\mathbf{a_{i+1}}$	…	$\mathbf{a_n}$

表的长度减少

```
Status ListDelete_Sq
                    (SqList &L, int i, ElemType &e) {
  if ((i < 1) || (i > L.length))  return ERROR;
                                        // 删除位置不合法
  p = &(L.elem[i-1]);      // p 为被删除元素的位置
  e = *p;                  // 被删除元素的值赋给 e
  q = L.elem+L.length-1;   // 表尾元素的位置
  for (++p; p <= q; ++p)  *(p-1) = *p;
                  //  被删除元素之后的元素左移
  --L.length;      // 表长减1
  return OK;
} // ListDelete_Sq
```

考虑移动元素的平均情况：

假设删除第i 个元素的概率为，q_i

则在长度为n 的线性表中删除一个元素所需移动元素次数的期望值为：

$$E_{dl} = \sum_{i=1}^{n} q_i (n-i)$$

若假定在线性表中任何一个位置上进行删除的概率都是相等的，则移动元素的期望值为：

$$E_{dl} = \frac{1}{n}\sum_{i=1}^{n}(n-i) = \frac{n-1}{2}$$

例如：ListDelete_Sq(L, 5, e)

```
p = &(L.elem[i-1]);
q = L.elem+L.length-1;
for (++p; p <= q; ++p)  *(p-1) = *p;
```

21	18	30	75	56	87				

§ 2.3
线性表的链式表示和实现

一、单链表

二、结点和单链表的 C 语言描述

三、线性表的操作在单链表中的实现

四、一个带头结点的单链表类型

一、单链表

用一组地址任意的存储单元存放线性表中的数据元素。

以元素(数据元素的映象)

+ 指针(指示后继元素存储位置)

= 结点

(表示数据元素 或 数据元素的映象)

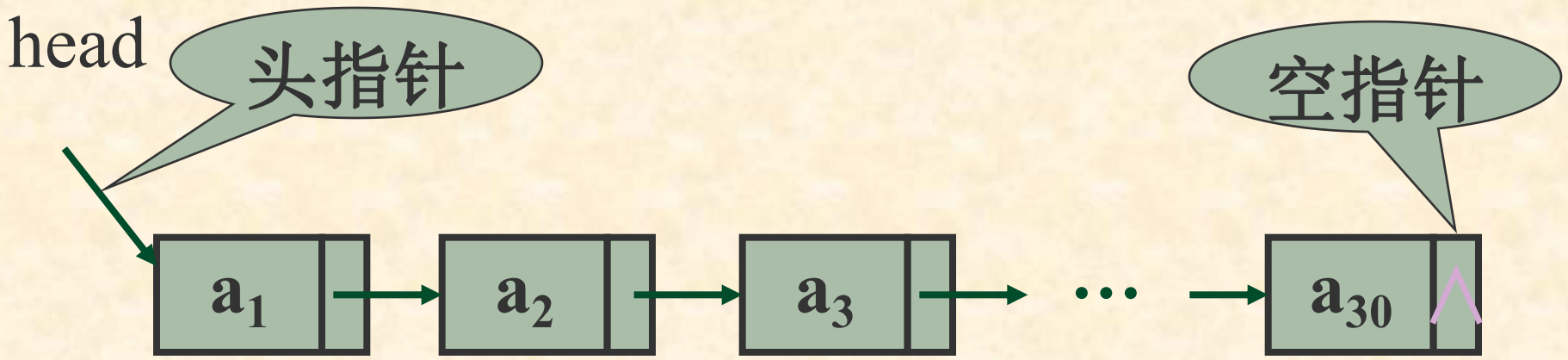
以“结点的序列”表示线性表

——称作链表

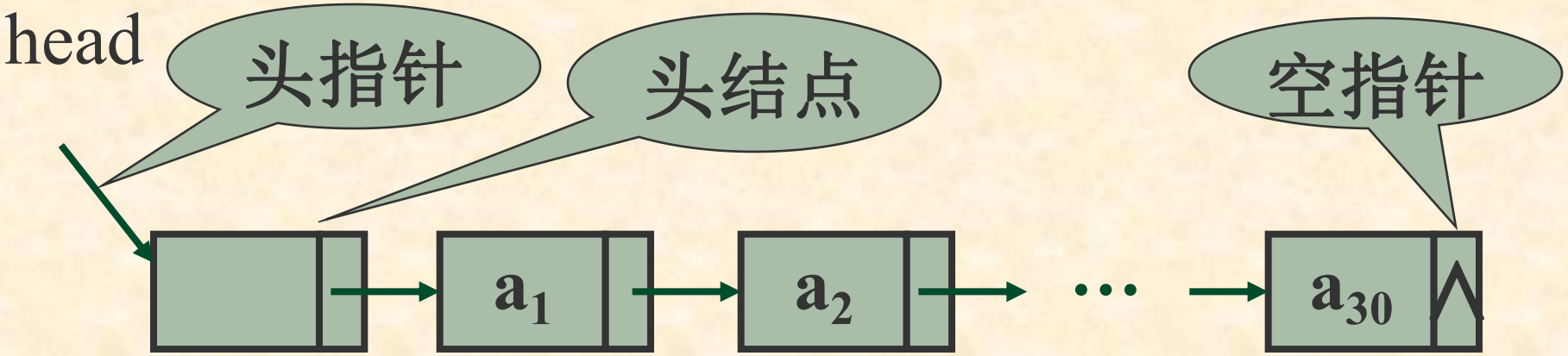
- 结点：数据元素的存储映象，由数据域和指针域构成。

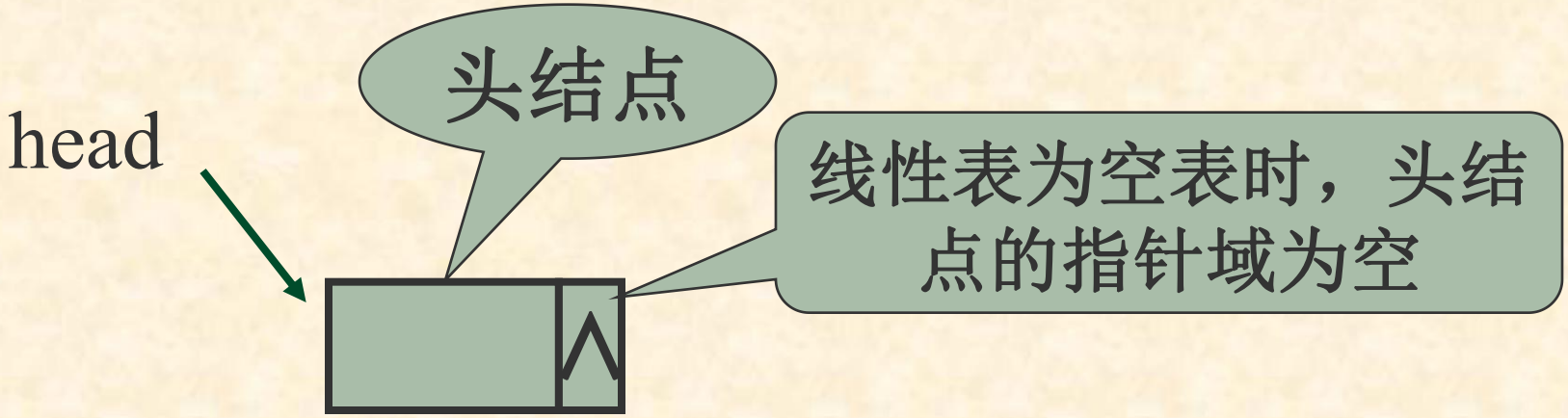
数据域	指针域

- **单链表**：一线性表的n个元素所对应的n个结点通过指针链接成一个**链表**。由于这种链表中每个结点只有一个指针域，故又称为**单链表**。



以线性表中第一个数据元素的存储地址作为线性表的地址，称作线性表的头指针。





有时为了操作方便，在第一个结点之前虚加一个“头结点”，以指向头结点的指针为链表的头指针。

链式存储表示的特点：

- 逻辑上相邻的元素对应的存储位置是通过指针反映的，不要求物理上相邻。进行插入、删除运算时，只需修改指针域。
- 一般不预先分配好足够的存储空间以备使用，当有元素时，需临时分配一个空的结点空间，填上信息，插入到线性链表中；当某个结点不再使用时，应将其存储空间释放。
- 每个结点都设有一个指针域，存储空间的开销较大。

二、结点和单链表的 C 语言描述

```
Typedef struct  LNode {
    ElemType      data;  // 数据域
    struct Lnode   *next;  // 指针域
} LNode, *LinkList;
```

LinkList L;　　// L 为单链表的头指针

结点

data next

三、单链表操作的实现

GetElem(L, i, e) // 取第i个数据元素

ListInsert(&L, i, e) // 插入数据元素

ListDelete(&L, i, e) // 删除数据元素

ClearList(&L) // 重置线性表为空表

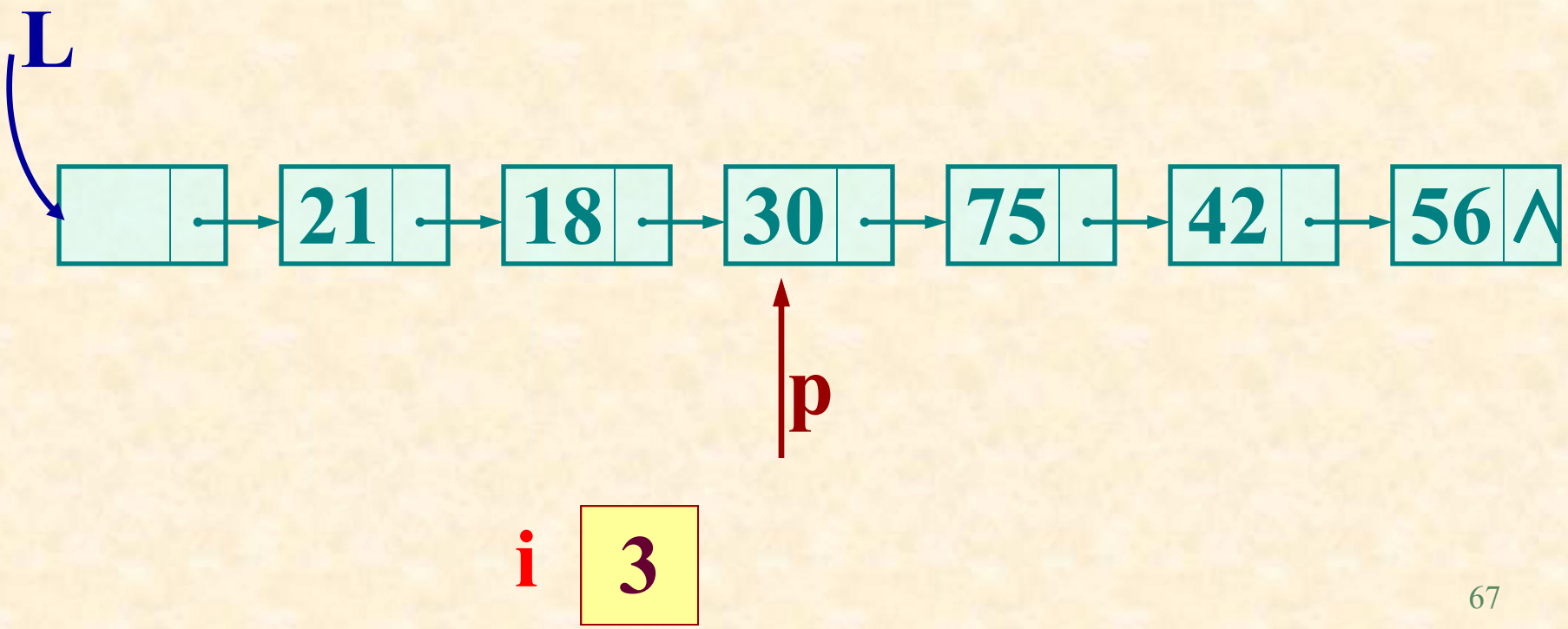
CreateList(&L, n)

// 生成含 ***n*** 个数据元素的链表

线性表的操作

GetElem(L, i, &e)

在单链表中的实现:



单链表是一种顺序存取的结构，为找第 i 个数据元素，必须先找到第 i-1 个数据元素。

因此，查找第 i 个数据元素的基本操作为：移动指针，比较 j 和 i 。

令指针 p 始终指向线性表中第 j 个数据元素。

```
Status GetElem_L(LinkList L, int i, ElemType &e) {
  // L是带头结点的链表的头指针，以 e 返回第 i 个元素
  p = L->next;  j = 1;  // p指向第一个结点，j为计数器
  while (p && j<i)  { p = p->next;  ++j;  }
      // 顺指针向后查找，直到 p 指向第 i 个元素
      // 或 p 为空
  if ( !p || j>i )
     return ERROR;        // 第 i 个元素不存在
  e = p->data;            // 取得第 i 个元素
  return OK;
} // GetElem_L
```

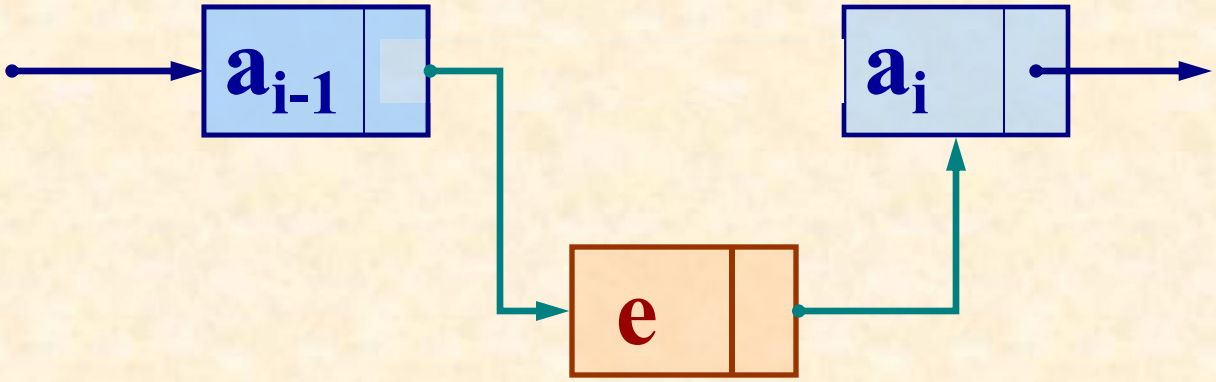
线性表的操作

ListInsert(&L, i, e)

在单链表中的实现：

有序对 $<a_{i-1}, a_i>$

改变为 $<a_{i-1}, e>$ 和 $<e, a_i>$



可见，在链表中插入结点只需要修改指针。但同时注意，若要在第 i 个结点之前插入元素，修改的是第 i-1 个结点的指针。

因此，在单链表中第 i 个结点之前进行插入的基本操作为:

找到线性表中第i-1个结点，然后修改其指向后继的指针。

```
Status ListInsert_L(LinkList L, int i, ElemType e) {
  // L 为带头结点的单链表的头指针，本算法
  // 在链表中第i 个结点之前插入新的元素 e
  p = L;    j = 0;
  while (p && j < i-1)
      { p = p->next;  ++j; }   // 寻找第 i-1 个结点
  if (!p || j > i-1)
      return ERROR;      // i 大于表长或者小于1
    ……
} // LinstInsert_L
```

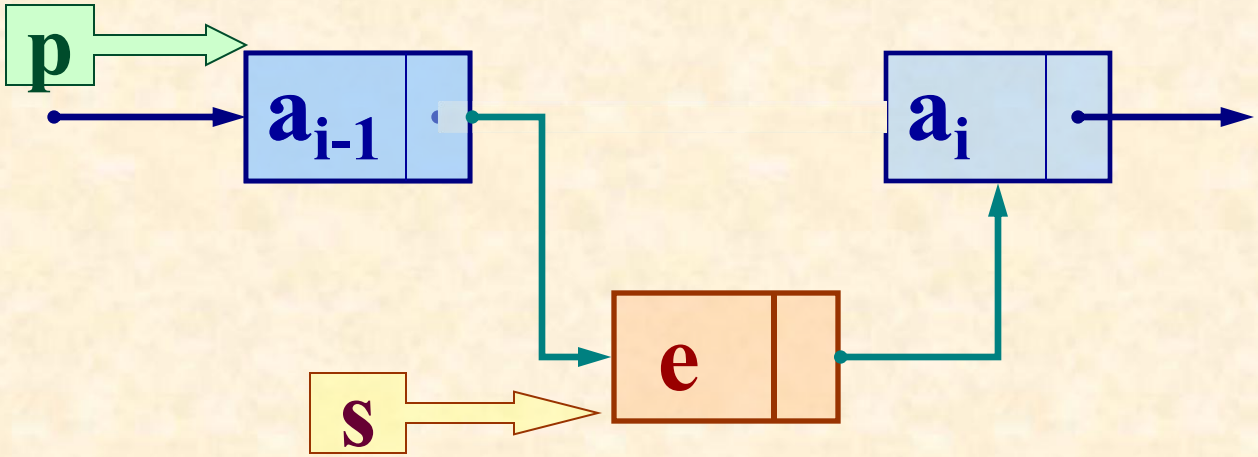
s = (LinkList) **malloc** (**sizeof** (LNode));

// 生成新结点

s->data = e;

s->next = p->next; p->next = s; // 插入

return OK;



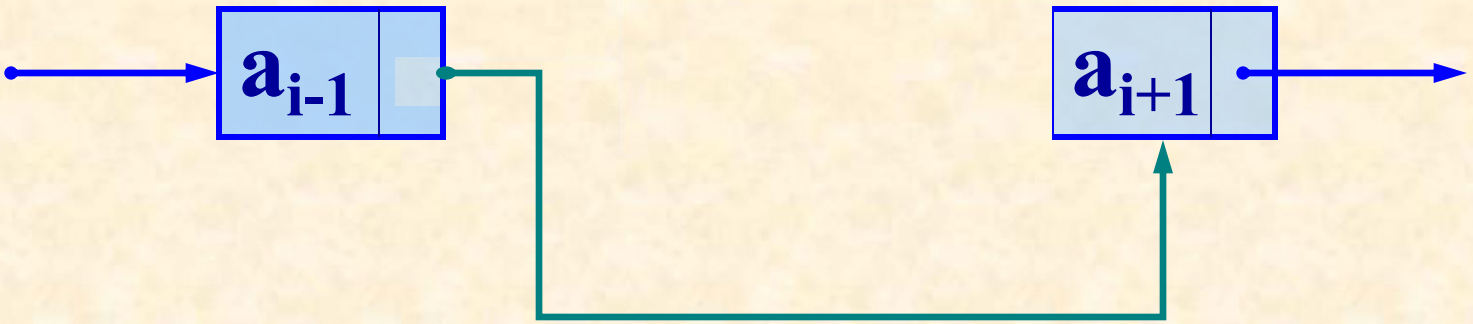
线性表的操作

ListDelete (&L, i, &e)

在链表中的实现:

有序对〈a_{i-1}, a_i〉和〈a_i, a_{i+1}〉

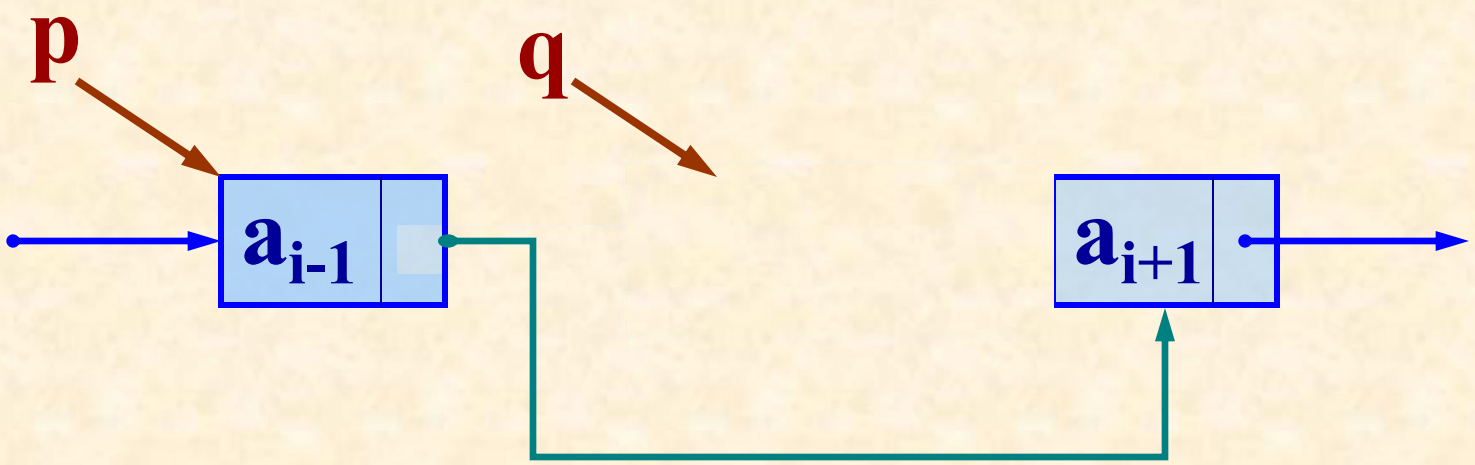
改变为〈a_{i-1}, a_{i+1}〉



在单链表中删除第 i 个结点的基本操作为:找到线性表中第i-1个结点，修改其指向后继的指针。

q = p->next; **p->next = q->next;**

e = q->data; ***free(q);***



```
Status ListDelete_L(LinkList L, int i, ElemType &e) {
  // 删除以 L 为头指针(带头结点)的单链表中第 i 个结点
    p = L;    j = 0;
    while (p->next && j < i-1) {  p = p->next;   ++j; }
                                  // 寻找第 i 个结点，并令 p 指向其前趋
    if  (!(p->next) || j > i-1)
        return ERROR;  // 删除位置不合理
    q = p->next;   p->next = q->next;  // 删除并释放结点
    e = q->data;   free(q);
    return OK;
} // ListDelete_L
```

操作 ClearList(&L) 在链表中的实现:

```
void ClearList(&L) {
  // 将单链表重新置为一个空表
   while (L->next) {
      p=L->next;    L->next=p->next;
      free(p);
   }
} // ClearList
```

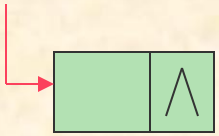
如何从线性表得到单链表？

链表是一个动态的结构，它不需要预分配空间，因此生成链表的过程是一个结点“逐个插入” 的过程。

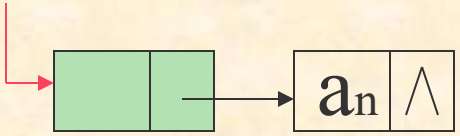
例如：逆位序输入 n 个数据元素的值，建立带头结点的单链表。

操作步骤：

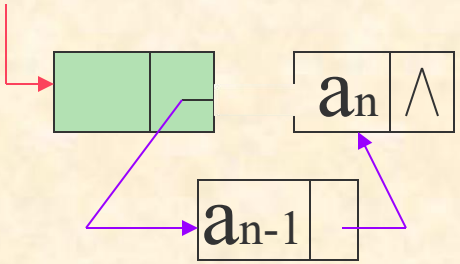
一、建立一个“空表”；



二、输入数据元素a_n，建立结点并插入；



三、输入数据元素a_{n-1}，建立结点并插入；



四、依次类推，直至输入a_1为止。

```
void CreateList_L(LinkList &L, int n) {
  // 逆序输入 n 个数据元素，建立带头结点的单链表
  L = (LinkList) malloc (sizeof (LNode));
  L->next = NULL;     // 先建立一个带头结点的单链表
  for (i = n; i > 0; --i) {
    p = (LinkList) malloc (sizeof (LNode));
    scanf(&p->data);    // 输入元素值
    p->next = L->next; L->next = p;  // 插入
  }
} // CreateList_L
```

用上述定义的单链表实现线性表的操作时，存在的问题：

1. 单链表的表长是一个隐含的值；
2. 在单链表的最后一个元素之后插入元素时，需遍历整个链表；
3. 在链表中，元素的“位序”概念淡化，结点的“位置”概念加强。

改进链表的设置：

1. 增加“表长”、“表尾指针” 和 “当前位置的指针” 三个数据域；
2. 将基本操作中的“位序 i ”改变为“指针 p”

2-2　作业

- 请分别写出在有、无头结点的单链表中实现如下操作：
- 1、在第一个结点前插入、删除
- 2、在链表中间插入、删除
- 3、在链表末尾插入、删除。

四、一个带头结点的线性链表类型

```
typedef struct LNode {  // 结点类型
    ElemType        data;
    struct LNode    *next;
} *Link, *Position;

Status MakeNode( Link &p, ElemType e );
  // 分配由 p 指向的值为e的结点，并返回OK，
  // 若分配失败，则返回 ERROR

void   FreeNode( Link &p );
  // 释放 p 所指结点
```

```
typedef struct {  // 链表类型
  Link  head, tail;     // 分别指向头结点和
                        // 最后一个结点的指针
  int   len;            // 指示链表长度
  Link  current;    // 指向当前被访问的结点
                    //的指针，初始位置指向头结点
} LinkList;
```

链表的基本操作：

{结构初始化和销毁结构}

Status InitList(LinkList **&**L);

// 构造一个空的线性链表 **L**，其头指针、

// 尾指针和当前指针均指向头结点，

// 表长为零。

Status DestroyList(LinkList **&**L);

// 销毁线性链表 **L**，**L**不再存在。

{引用型操作}

Status ListEmpty (LinkList L); //判表空

int ListLength(LinkList L); // 求表长

Status Prior(LinkList L);

// 改变当前指针指向其前驱

Status Next (LinkList L);

// 改变当前指针指向其后继

ElemType GetCurElem (LinkList L);

// 返回当前指针所指数据元素

Status LocatePos(LinkList L, int i);

// 改变当前指针指向第i个结点

Status LocateElem (LinkList L, ElemType e,

Status (*compare)(ElemType, ElemType));

// 若存在与e 满足函数compare( )判定关系的元

// 素，则移动当前指针指向第1个满足条件的

// 元素的前驱，并返回OK; 否则返回ERROR

Status ListTraverse(LinkList L, **Status**(*visit)());

// 依次对L的每个元素调用函数visit()

{加工型操作}

Status ClearList (LinkList **&**L);

// 重置 L 为空表

Status SetCurElem(LinkList **&**L, ElemType e);

// 更新当前指针所指数据元素

Status Append (LinkList **&**L, Link s);

// 在表尾结点之后链接一串结点

Status InsAfter (LinkList **&**L, Elemtype e);

// 将元素 e 插入在当前指针之后

Status DelAfter (LinkList **&**L, ElemType**&** e);

// 删除当前指针之后的结点

```
Status InsAfter( LinkList& L, ElemType e ) {
 // 若当前指针在链表中，则将数据元素e插入在线性链
 // 表L中当前指针所指结点之后，并返回OK;
 // 否则返回ERROR。
    if ( ! L.current )  return ERROR;
    if (! MakeNode( s, e) )  return ERROR;
    s->next = L.current->next;
    L.current->next = s;
    if (L.tail = =L.current)  L.tail = s;
    L.current = s;    return OK;
} // InsAfter
```

```
Status DelAfter( LinkList& L, ElemType& e ) {
  // 若当前指针及其后继在链表中，则删除线性链表L中当前
  // 指针所指结点之后的结点，并返回OK; 否则返回ERROR。
    if ( !(L.current && L.current->next ) )
        return ERROR;
    q = L.current->next; //q为要删除的节点
    L.current->next = q->next;
    if (L.tail = =q)  L.tail = L.current;
    e=q->data;  FreeNode(q);
    return OK;
} //DelAfter
```

利用上述定义的线性链表如何完成线性表的其它操作？

例一

```
Status ListInsert_L(LinkList L, int i, ElemType e) {
  // 在带头结点的单链线性表 L 的第 i 个元素之前插入元素 e
  if (!LocatePos (L, i-1))  return ERROR;
                    // i 值不合法，第 i-1 个结点不存在
  if (InsAfter (L, e))  return OK;      // 完成插入
  else return ERROR;
} // ListInsert_L
```

• 静态链表：

链结点空间的管理方法

- 静态数据结构

  顺序表，在程序的运行期间数组的大小不再变化

- 动态数据结构

  链表，在程序的运行期间大小可自由变化

对于链表使用的空间，通常可用系统中提供的动态存储分配函数（例如C语言中的函数malloc和free），由系统动态管理。

• 静态链表示例：

用数组实现的链式结构，称为静态链表

0		4
1	a_4	0
2		6
3	a_3	1
4		5
5		7
6	a_1	8
7		9
8	a_2	3
9		10
10		0

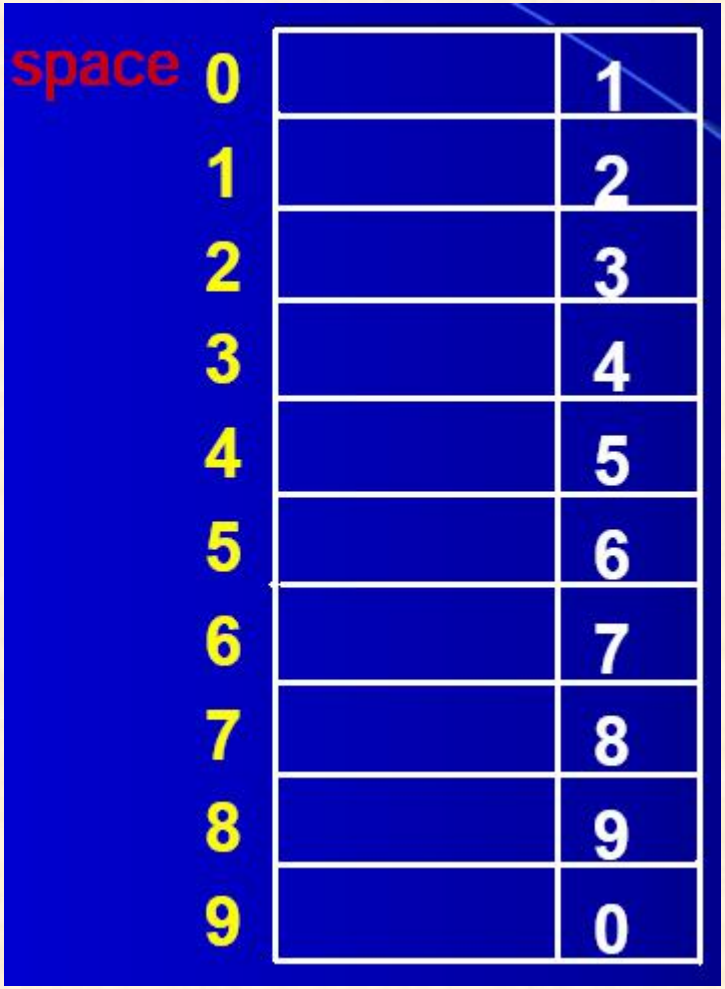
← 备用空间链表

← L＝2

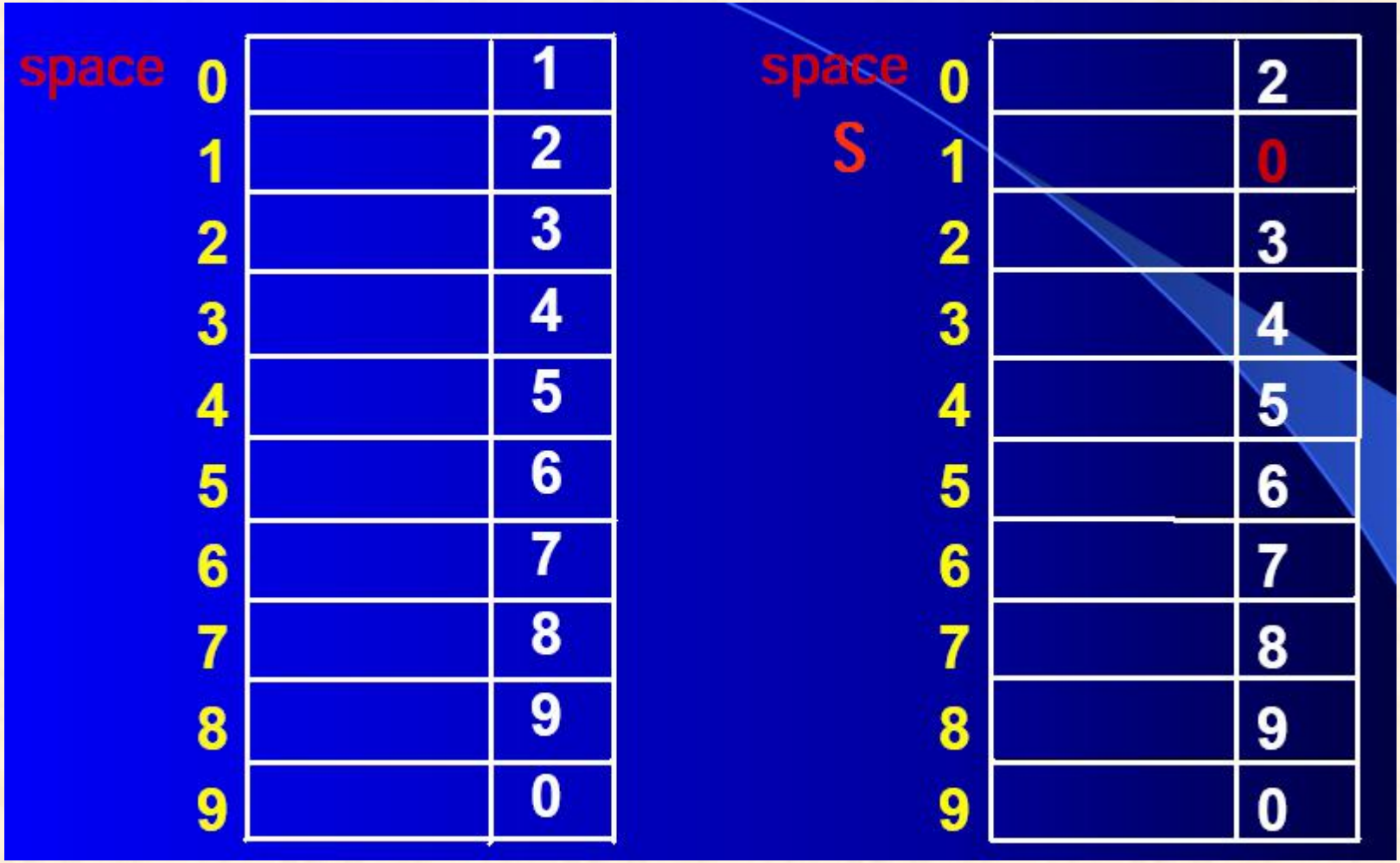
静态链表的类型定义

```
#define MAXSIZE 1000   // 链表的最大长度
typedef struct{
    ElemType    data;
    int cur;
} component, SLinkList[MAXSIZE];
```

备用空间链表



必须自己实现 free和 malloc操作


space
0 1
1 2
2 3
3 4
4 5
5 6
6 7
7 8
8 9
9 0
space
S
0 2
1 0
2 3
3 4
4 5
5 6
6 7
7 8
8 9
9 0

建空链表

静态链表的插入

插入wang前　　插入wang后

静态链表的删除

删除sun前　　删除sun后

建备用空间链表

```
void InitSpace_SL(SLinkList &space)
{ //建备用空间链表，  space为头指针，0为空指针
   for(i=0;i< MAXSISE-1; ++i)
        space[i].cur=i+1;
   space[MAXSISE-1 ].cur=0;
} // InitSpace_SL
```

从备用链表中获取一个结点

```
Int Malloc_SL(SLinkList &space)
{ //若备用空间链表非空，返回分配结点的下标，否则返回0
    if (!space[0].cur) return 0;
    i=space[0].cur;
    space[0].cur=space[i].cur;
    return i;
} //Malloc_SL
```

将结点回收到备用链表中

```
void Free_SL(SLinkList &space,int k)
{ //将下标为k的结点回收到备用空间链表（头）
    space[k].cur=space[0].cur;
    space[0].cur=k;
} //Free_SL
```

静态链表的插入算法

```
Status ListInsert_SL(SLinkList &space,int
S, int i,ElemTypa e)//在第i个节点后插入e
{ //S为静态链表头结点下标
   k=S; j=0;
   while (k&&j<i-1){k=space[k].cur ; ++j;}
                    //寻找第i-1个元素结点
   if(!k||j>i-1)return ERROR;
   m=Malloc_SL(space);        // 分配新结点
   space[m].data=e;
   space[m].cur= space[k].cur ;     //插入新结
   space[k].cur=m ;
   return OK;
}
```

• 单链表与顺序表的比较：

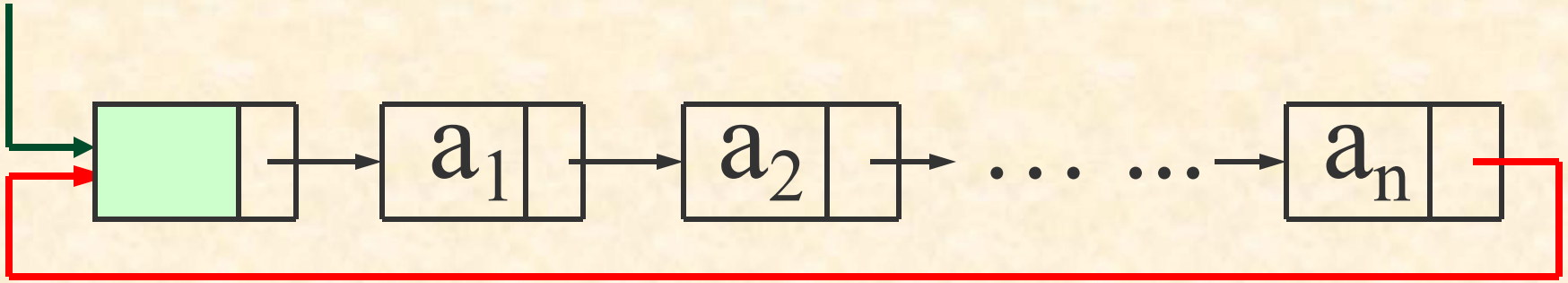
(1) 单链表的存储密度比顺序表低；但在许多情况下链式的分配比顺序分配有效 ；

(2) 在单链表里进行插入、删除运算比在顺序表里容易得多；

(3) 对于顺序表，可随机访问任一个元素，而在单链表中，需要顺着链逐个进行查找。

• 循环链表

最后一个结点的指针域的指针又指回第一个结点的链表



和单链表的差别仅在于，判别链表中最后一个结点的条件不再是“后继是否为空”，而是“后继是否为头结点”。

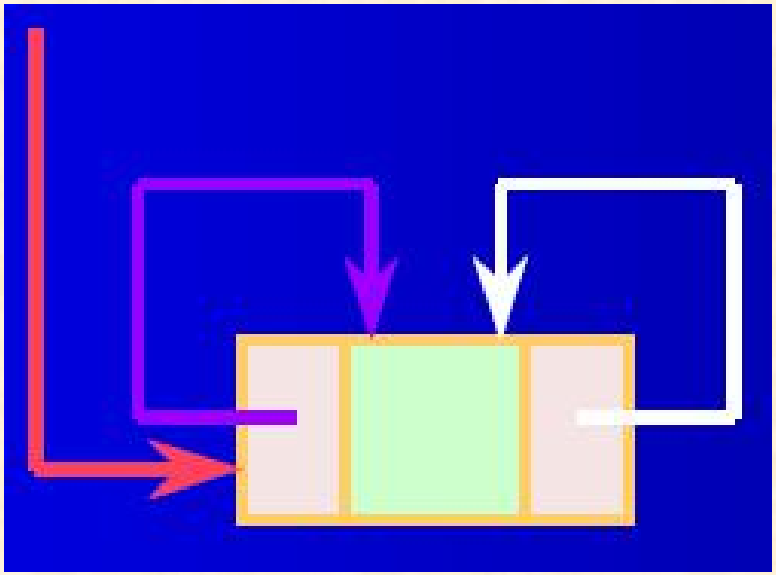
双向链表

prior	data	next

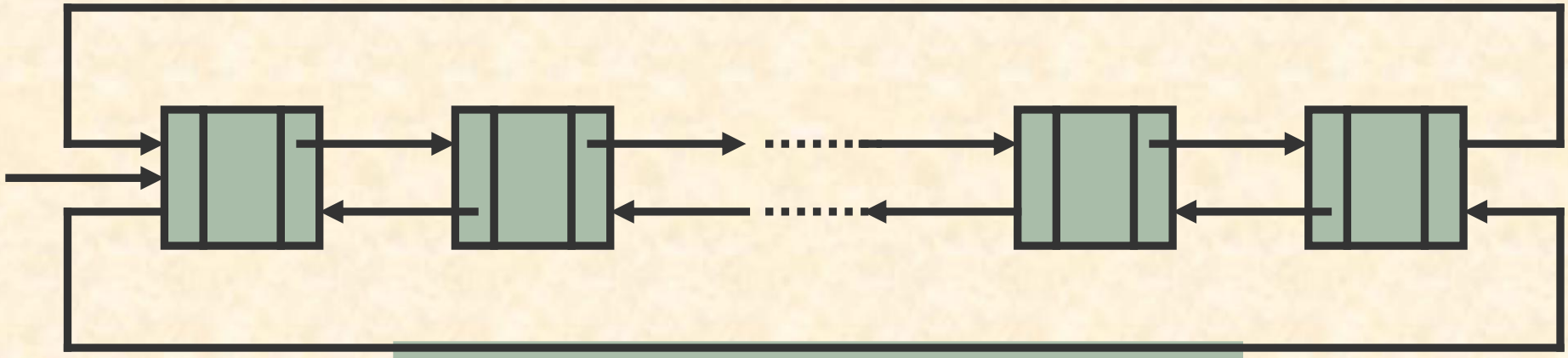
```
typedef struct DuLNode {
    ElemType    data;    // 数据域
    struct DuLNode   *prior;
                    // 指向前驱的指针域
    struct DuLNode   *next;
                    // 指向后继的指针域
} DuLNode, *DuLinkList;
```

双向循环链表

空表



非空表



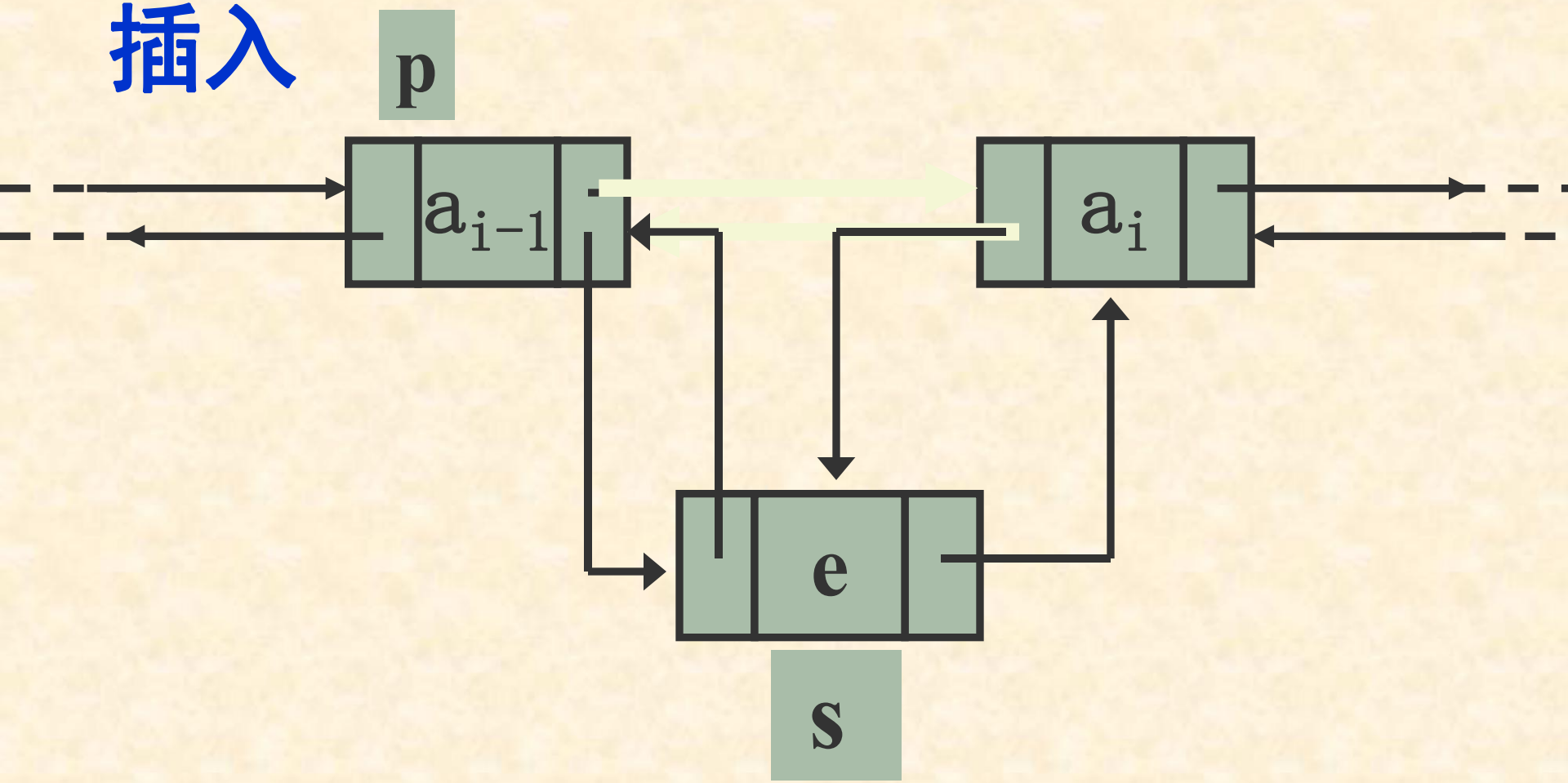
带头结点的双向循环链表

双向链表的操作特点：

1. “查询” 和单链表相同。

2. “插入” 和“删除”时需要同时修改两个方向上的指针。

插入



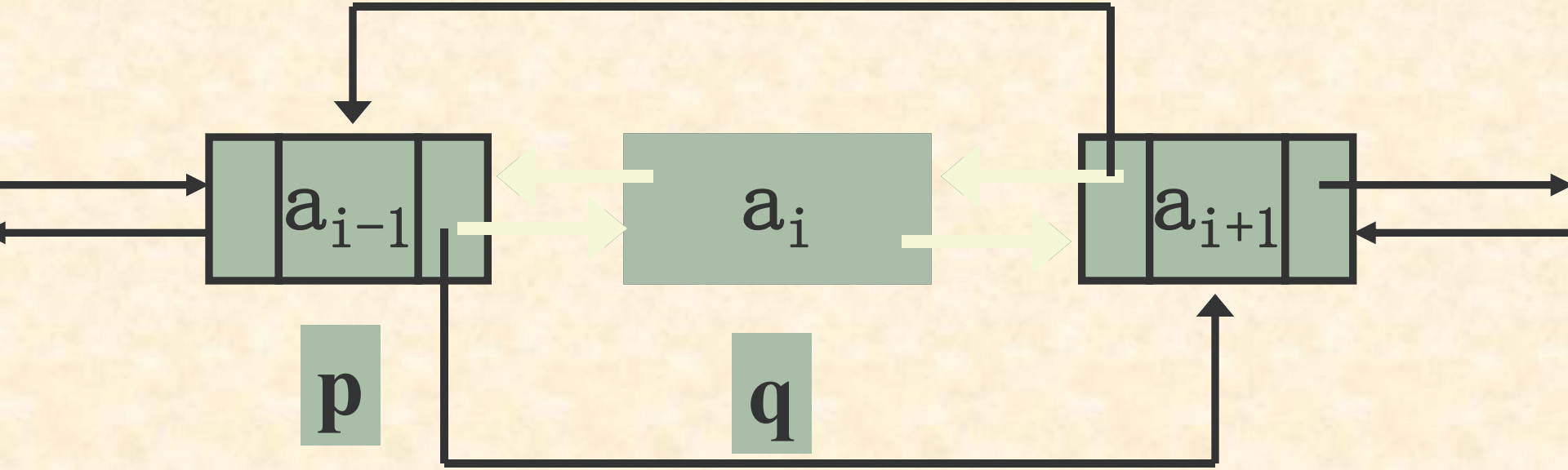
s->next = p->next;

p->next = s;

s->next->prior = s;

s->prior = p;

删除




```
q = p->next
p->next = q->next;
p->next->prior = p;
free(q);
```

§2.4 一元多项式的表示及相加

在计算机中，可以用一个线性表来表示：

$$P = (p_0, p_1, \ldots, p_n)$$

但是对于形如

$$S(x) = 1 + 3x^{10000} - 2x^{20000}$$

的多项式，上述表示方法是否合适？

一般情况下的一元稀疏多项式可写成

$$P_n(x) = p_1x^{e1} + p_2x^{e2} + --- + p_mx^{em}$$

其中：p_i 是指数为e_i 的项的非零系数，

$$0 \le e_1 < e_2 < --- < e_m = n$$

可以下列线性表表示：

$$（（p_1, e_1）, (p_2, e_2), ---, (p_m,e_m)）$$

例如:

$$P_{999}(x) = 7x^3 - 2x^{12} - 8x^{999}$$

可用线性表

$$((7, 3), (-2, 12), (-8, 999))$$

表示

抽象数据类型一元多项式的定义如下：

ADT Polynomial {

数据对象：

D＝{ a_i | a_i ∈TermSet, i=1,2,...,m, m≥0

TermSet 中的**每个元素包含一个**

表示系数的实数和表示指数的整数 }

数据关系：

R1＝{ <a_{i-1} ,a_i >|a_{i-1} ,a_i∈D, i=2,...,n

且$\mathbf{a_{i-1}}$**中的指数值**＜$\mathbf{a_i}$**中的指数值** }

基本操作：

CreatPolyn (&P, m)

操作结果：输入 m 项的系数和指数，建立一元多项式 P。

DestroyPolyn (&P)

初始条件：一元多项式 P 已存在。

操作结果：销毁一元多项式 P。

PrintPolyn (&P)

初始条件：一元多项式 P 已存在。

操作结果：打印输出一元多项式 P。

PolynLength(P)

初始条件：一元多项式 P 已存在。

操作结果：返回一元多项式 P 中的项数。

AddPolyn (&Pa, &Pb)

初始条件：一元多项式 Pa 和 Pb 已存在。

操作结果：完成多项式相加运算，即：

Pa = Pa＋Pb，并销毁一元多项式 Pb。

SubtractPolyn (&Pa, &Pb)

… …

} ADT Polynomial

一元多项式的实现：

```
typedef  OrderedLinkList   polynomial;
  // 用带表头结点的有序链表表示多项式
```

结点的数据元素类型定义为：

```
typedef struct {        // 项的表示
    float  coef;        // 系数
    int    expn;        // 指数
} term, ElemType;
```

```
Status CreatPolyn ( polynomail &P, int m ) {
  // 输入m项的系数和指数，建立表示一元多项式的有序链表P

  InitList (P); h=Getheat(P); // 设置头结点的数据元素
  e.coef = 0.0;  e.expn = -1; SetCurElem (h, e);
    for ( i=1; i<=m; ++i ) {  // 依次输入 m 个非零项
        scanf (e.coef, e.expn);
        if (!LocateElem ( P, e, q, (*cmp)( )) )
            if ( MakeNode ( s, e ) )   InsFirst(q,s);
    }
    return OK;
} // CreatPolyn
```

注意：1. 输入次序不限；

2. 指数相同的项只能输入一次。

- 在用线性链表来存储一个多项式时，多项式中的每个系数非零项对应一个具有三个域的结点，结点的结构如下：

coef存放系数， exp存放指数， link域存放指向下一项所在结点的指针。

- 例如：$A(x)=x^{2000}+12x^{1000}+5$可以表示成以下的形式：

假设Bm(x)为一元m阶多项式，

则Bm(x)与An(x)的相加运算

Cn(x)= An(x) +Bm(x)用线性表表示为

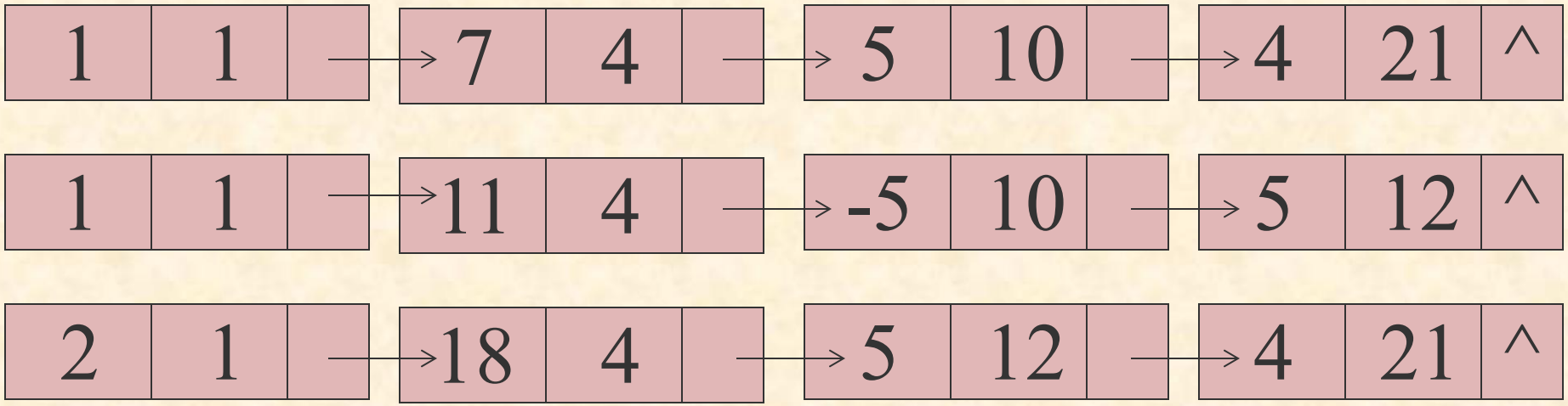
$C=(a_n, a_{n-1}, \ldots, a_{m+1}, a_m+b_m, a_{m-1}+b_{m-1}, \ldots, a_0+b_0)$

上式中假设n>m

$$A(x)=x + 7x^4 + 5x^{10} + 4x^{21}$$

$$B(x)= x + 11x^4 - 5x^{10} + 5x^{12}$$

$$A(x)= 2x +18x^4+ 5x^{12}+4x^{21}$$



- 一元多项式加法运算：

　　两个多项式中所有指数相同的项对应系数相加，若和不为零，则构成和多项式中的一项，而所有指数不相同的那些项均复制到“和多项式”中。

- 算法中可设置两个活动指针变量p和q，它们分别沿着A链表与B链表依次访问链表中的结点，p的初值为A，q的初值为B，即分别指向各自链表的第一个结点。

- **算法的核心：**

比较p和q所指结点的exp域的值，若相同，则将两结点的系数域值相加，若相加的结果不为0，则把这个结果和相应指数分别存入C链表中新分配的空结点的系数域和指针域；若p与q所指的结点指数不同，先将指数较低的那一项复制到C链表中，然后将p或者q移向下一个结点；重复上述步骤，当p=NULL（或q=NULL）时，就将B链表（或A链表）中剩余的部分复制到C链表中，直到p、q均为空时算法结束。

Status AddPolyn (polynomial **&**Pa, polynomial **&**Pb)

{

// **利用两个多项式的结点构成“和多项式”** Pa = Pa + Pb

算法2. 22(2. 23) … …

… …

} // AddPolyn

本章小结

1. 了解线性表的逻辑结构特性是数据元素之间存在着线性关系，在计算机中表示这种关系的两类不同的存储结构是顺序存储结构和链式存储结构。用前者表示的线性表简称为顺序表，用后者表示的线性表简称为链表。

2. 熟练掌握这两类存储结构的描述方法，以及线性表的各种基本操作的实现。

本章作业

1. 在什么情况下用顺序表比链表好？

2. 已知L是带表头结点的非空单链表，且P结点既不是首元结点，也不是尾元结点，试从下列提供的答案中选择合适的语句序列。

a. 删除P结点的直接后继结点的语句序列是_____。

b. 删除P结点的直接前驱结点的语句序列是_____。

c. 删除P结点的语句序列是_____。

d. 删除首元结点的语句序列是_____。

e. 删除尾元结点的语句序列是_____。

(1) p = p->next;

(2) p->next = p;

(3) p->next = p->next->next;

(4) p = p->next->next;

(5) while (p!=NULL) p=p->next;

(6) while(q->next!=NULL) { p=q; q=q->next;

(7) while (p->next!=q) p=p->next;

(8) while (p->next->next!=q) p=p->next;

(9) while (p->next->next!=NULL) p=p->next;

(10) q=p; 11) q=p->next; 12) p=l;

3. 算法设计。设顺序表va中的数据元素递增有序，请设计一算法，将x插入到顺序表的适当位置，以保持该表的有序性。

4. 算法设计。请设计一个算法，实现顺序表的原地逆置，即利用原表的存储空间将线性表（a1，a2，……，an）逆置为（an，……，a2，a1）。

5. 算法设计。请设计一个算法，实现单链表的原地逆置。

上机实验一

1、采用单向环表实现约瑟夫环。

请按以下要求编程实现：

① 从键盘输入整数m，通过create函数生成一个具有m个结点的单向环表。环表中的结点编号依次为1，2，……，m。

② 从键盘输入整数s（1<=s<=m）和n，从环表的第s个结点开始计数为1，当计数到第n个结点时，输出该第n结点对应的编号，将该结点从环表中消除，从输出结点的下一个结点开始重新计数到n，这样，不断进行计数，不断进行输出，直到输出了这个环表的全部结点为止。

例如，m=10，s=3，n=4。则输出序列为：6，10，4，9，5，2，1，3，8，7。

上机实验一

2、归并顺序表（选作）。

请按以下要求编程实现：

① 从键盘输入两个升序排列的整数序列linka和linkb，每个序列以输入0为结束标记。

② 将链表linka和linkb归并为linkc，linkc仍然为升序排列。归并完成后，linka和linkb为空表。输出linkc。

③ 对linkc进行处理，保持升序不变，删除其中重复的整数，对重复的整数只保留一个，输出删除重复整数后的链表。

上机实验一

例如：linka输入为：10　20　30　40　50　0

linkb输入为：15　20　25　30　35　40　45　50　0

归并后的linkc为：10　15　20　20　25　30　30　35　40　40　45　50　50

删除重复后的linkc为：10　15　20　25　30　35　40　45　50

作业1

概念：

元首结点： 链表中存储线性表中的第一个元素的结点。

头结点： 在链表的元首结点之前附加的一个结点。在单链表中设置头结点的作用是插入和删除元首结点时不必进行特殊处理。

头指针： 指向链表中的第一个结点（或为头结点或为元首结点）的指针

手工作业

1. 描述以下三个概念的区别：头指针、头结点、首元结点。

2. 在单链表中，除了首元结点外，任一结点的存储位置由_____指示。

3. 在单链表中设置头结点的作用是_____。

4. 画出执行下列各行语句后各指针及链表的示意图。

```
L = (LinkList)malloc(sizeof(LNode)); P=L;
For (i = 1; i <=4; i++) {
    P->next = (LinkList) malloc (sizeof(Lnode));
    P = P ->next; P->data = i * 2 –1;
}
P->next = NULL;
For (i = 4; i >=1; i--;) ListInsert_L(L, i + 1, i * 2);
For ( i = 1; i <= 3; i++) ListDelete_L(L, i , e);
```

5. 简述以下算法的功能。

(1)
```
Status A(LinkList &L) { //L是无表头结点的单链表
    if (L && L->next) {
        Q = L; L = L->next; P = L;
        while (P->next) P = P->next;
        P->next = Q; Q->next = NULL;
    }
    return OK;
}
```

(2)
```
void BB(LNode *s, LNode *q) {
    p = s;
    while (p->next != q) p = p->next;
    p->next = s;
  }
void AA(LNode *pa, LNode *pb) {
    //pa和pb分别指向单循环链表中的两个结点
    BB(pa, pb);
    BB(pb, pa);
}
```

编程作业

1. 在dslib中，实现LinkList的定义。

2. 试写一算法，不用基本操作，实现顺序表的就地逆置，即利用原表的存储空间逆置。

3. 试写一算法，不用基本操作，对无头结点单链表实现就地逆置。